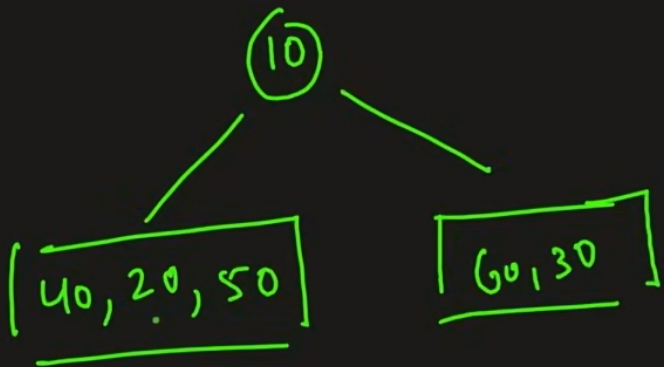


Construct Binary Tree from Inorder & Pre order



(Left Root Right)
inorder → [40 20 50] 10 [60 30]

preorder → [10] 20 40 50 30 60
(Root Left Right)

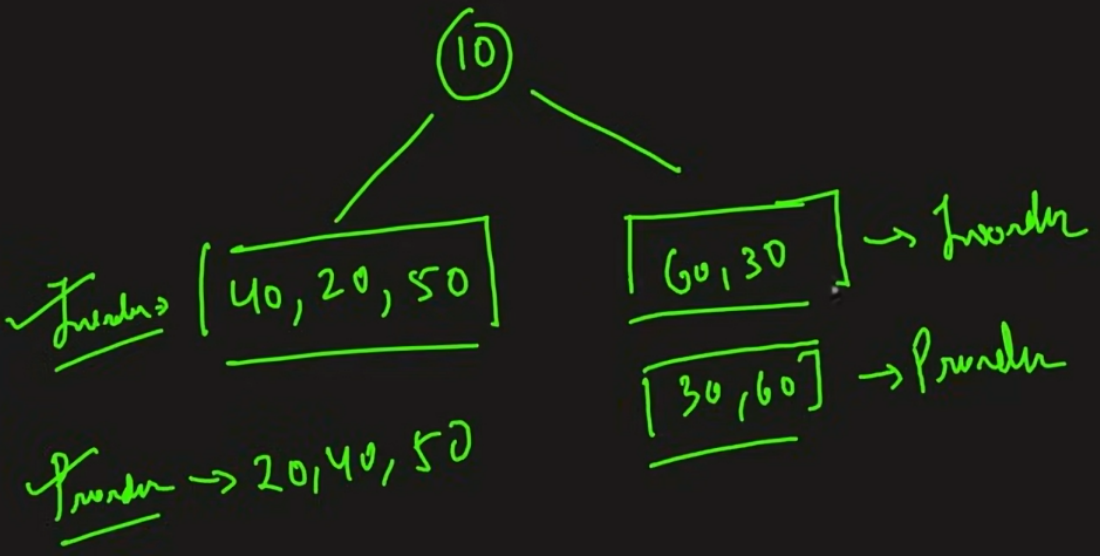
Construct Binary Tree from Inorder { Pre order

(Left Root Right)

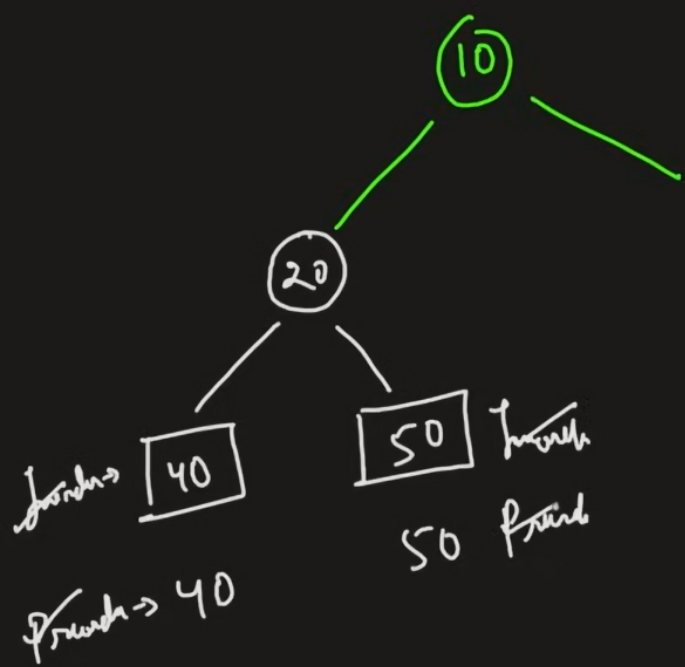
inorder → [40 20 50 | 10 | 60 30]

preorder → [10 | 20 40 50 | 30 60]

(Root Left Right)



Construct Binary Tree from Inorder & Pre order

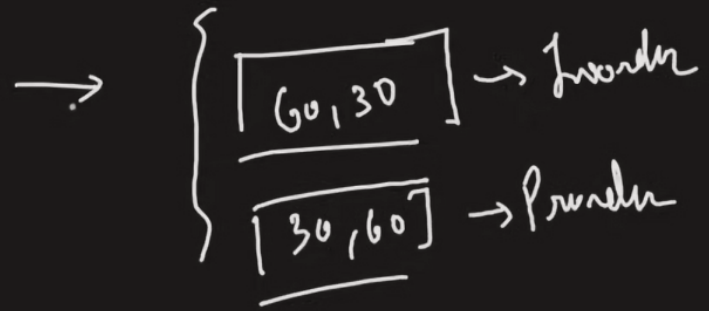
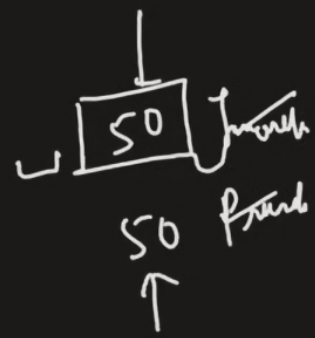
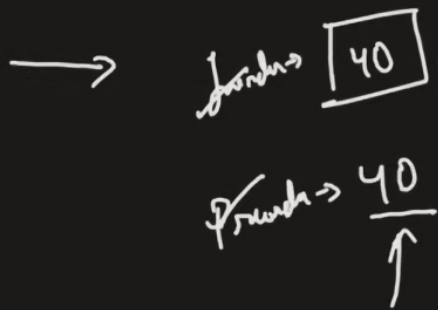
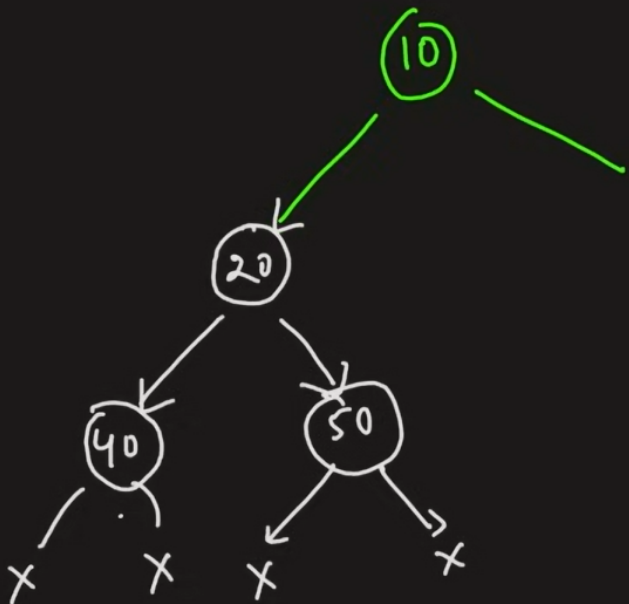


Left Root Right
Inorder $\rightarrow$ [40, 20, 50]

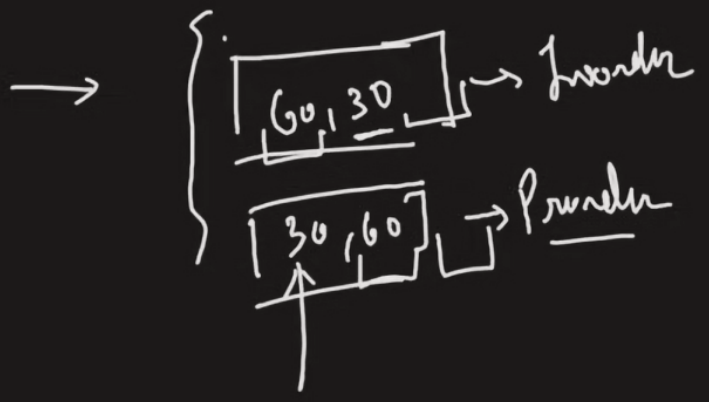
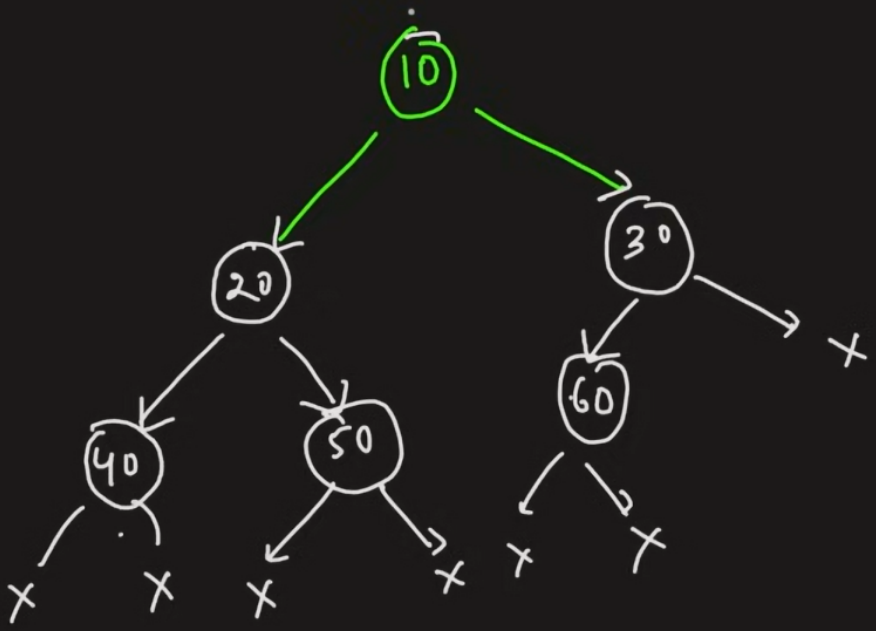
Preorder $\rightarrow$ 20, 40, 50
Root Left Right

$\left\{ \begin{array}{l} [60, 30] \rightarrow \text{Inorder} \\ [30, 60] \rightarrow \text{Preorder} \end{array} \right.$

Construct Binary Tree from Inorder & Pre order



Construct Binary Tree from Inorder & Preorder



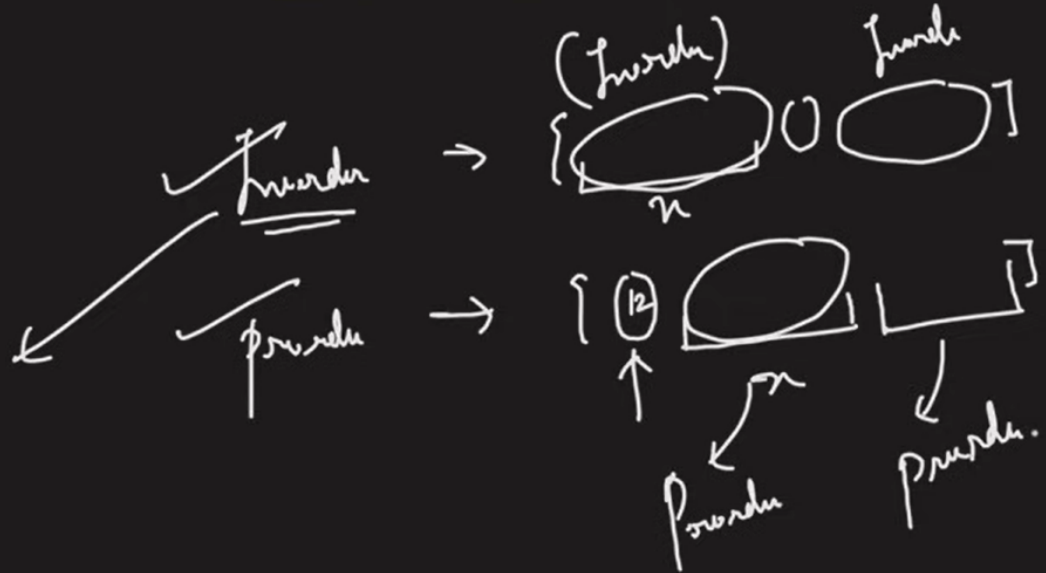
Inorder → 60

Preorder → 30
↑

Construct Binary Tree from Inorder & Preorder

Code

Hash

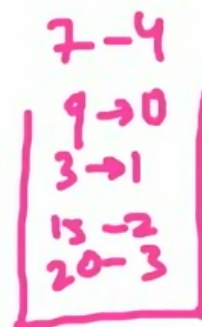
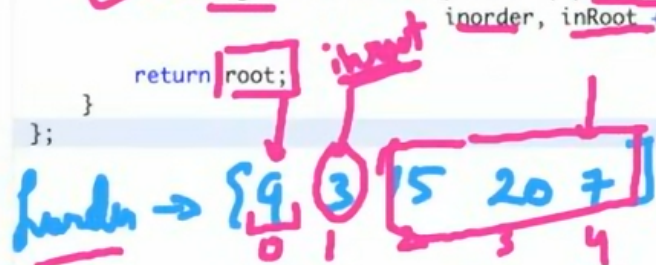
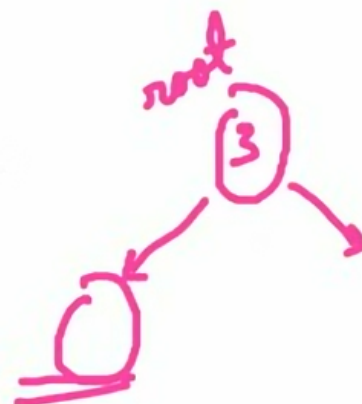


L34. Construct a Binary Tree from Preorder and Inorder Traversal | C++ | Java

```

1
2 class Solution {
3 public:
4     TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
5         map<int, int> inMap;
6
7         for(int i = 0; i < inorder.size(); i++) {
8             inMap[inorder[i]] = i;
9         }
10
11         TreeNode* root = buildTree(preorder, 0, preorder.size() - 1,
12                                     inorder, 0, inorder.size() - 1, inMap);
13         return root;
14     }
15     TreeNode* buildTree(vector<int>& preorder, int preStart, int preEnd,
16                         vector<int>& inorder,
17                         int inStart, int inEnd, map<int, int> inMap) {
18         if(preStart > preEnd || inStart > inEnd) return NULL;
19
20         TreeNode* root = new TreeNode(preorder[preStart]);
21
22         int inRoot = inMap[root->val];
23         int numsLeft = inRoot - inStart;
24
25         root->left = buildTree(preorder, preStart + 1, preStart + numsLeft,
26                             inorder, inStart, inRoot - 1, inMap);
27         root->right = buildTree(preorder, preStart + numsLeft + 1, preEnd,
28                               inorder, inRoot + 1, inEnd, inMap);
29
30         return root;
31     }
32 };

```



Your previous code was restored from your local storage. [Reset to default](#)

Console - Contribute /

Run Code ^

Submit